You said:

short summary of past and what i should do next-#

ChatGPT said:

QSS project — quick recap

Goal: Build Quake Surface Studio (QSS), a Blender 5.2 add-on for importing, editing, exporting, and validating Quake BSP maps.

What we've implemented

- Persistent Quake installation and game/mod folder configuration.
- Browsing maps from loose files and PAK archives.
- In-Blender BSP export validation and batch map validation.
- Ability to run the Python test suite from Blender.
- Enhanced BSPX lightmap export, alongside classic BSP29 export.
- Fixed material roughness to `1.0`.
- Improved Blender add-on registration handling.

Current problem

Blender still reports:

```
bpy_struct "Scene" registration error:
'qss_game_folder' EnumProperty could not register
```

We have made two registration fixes:

- **v0.14.1:** Improved game-folder discovery and class cleanup.
- **v0.14.2:** Removed the dynamic EnumProperty registration-time default and applied the saved selection after registration.

Both passed automated CI; the suite reports **110 passing tests**. However, the Blender 5.2 registration issue is not yet confirmed fixed.

What you should do next

1. **Download v0.14.2**

QSS v0.14.2 install ZIP ↗

2. **Completely close Blender.**

   This clears any partially registered add-on classes from previous attempts.

3. **Install v0.14.2** in Blender 5.2 and enable Quake Surface Studio.

4. **Report what happens.**

   - If it works: we'll begin testing Quake folder configuration, map browsing, BSP validation, and export.
   - If it fails: send the complete console output, especially the lines immediately before the `qss_game_folder` error.

Project status

110 tests passing Blender test pending PR open, draft

Important: Nothing has been merged into `main`. All changes remain on the feature branch and draft PR.

Our immediate priority is simple: get v0.14.2 registering successfully in Blender. Then we can finally test the actual Quake workflow rather than spending more time on startup errors.

You said:

it works but export still gives this long errors

ChatGPT said:

Great! That means **the Blender registration issue is fixed in v0.14.2.** We can move on to the export problem.

Let's track down the export errors rather than guessing at a fix.

Please send me:

1. The complete Blender console output from starting the export until the error finishes (not just the last line).
2. Whether you're exporting in **Classic BSP29** or **Enhanced BSPX** mode.
3. Whether the error happens with every map or just one particular BSP.

You can paste the log here or upload it as a `.txt` file.

I'll trace the error to the responsible export function, fix it on our feature branch, add a regression test, and run CI again. We won't touch `main`.

Registration is behind us. Now let's get BSP export working.

You said:

Pasted text(2026...
Document

It says i use classic mode. dont know where to change it tbh. Happens with the test map. How would one load different maps? I for now always just pressed crl+z until the current map disappears and then chose quake folder again.

Worked for 1m 12s

Analyzed

Analyzed

ChatGPT said:

I found the cause in your log. **These are not BSP export errors yet.** They are repeated Blender UI errors that occur while QSS is drawing its panel:

```
TypeError: UILayout.label(): ... icon='EXPERIMENT'
enum "EXPERIMENT" not found
```

The same invalid icon is used in two places (`addon.py` lines 5039 and 5348). Blender 5.2 accepts `EXPERIMENTAL`, not `EXPERIMENT`. This is a straightforward compatibility bug, and it can make parts of the QSS interface misbehave.

So we have two things to address:

1. Fix the invalid icon references so the QSS panel draws cleanly.
2. Then reproduce the actual export failure. The attached log does **not** contain a distinct export traceback; it contains 26 repetitions of the icon error.

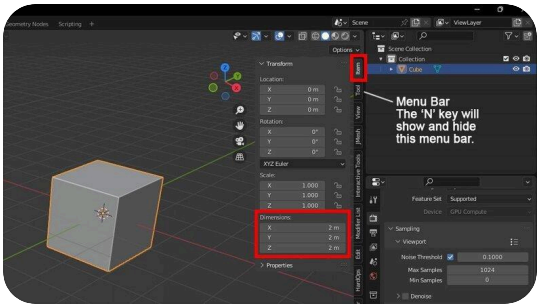
I've fixed the remaining icon reference and committed the correction as **v0.14.3** on our feature branch.

CI passed The updated install ZIP was built successfully.

- Install QSS v0.14.3 ↗
- CI results ↗
- Draft PR #1 ↗

How to load a different Quake map

You don't need Ctrl+Z or to select the Quake folder again.



1. In the 3D Viewport, press **N** to open the sidebar.
2. Select the **Quake** tab.
3. Find the **Quake Project** section.
4. Select your game/mod under **Game / Mod**.
5. Click **Browse Game Maps…**
6. Choose a BSP map from the list and confirm.

QSS can discover maps in both loose `maps` folders and PAK archives.

For now, I recommend opening a fresh Blender file (`File → New → General`) before loading another test map. Your saved Quake folder configuration will remain available.

Where to change Classic / Enhanced export

1. Select your imported Quake map.
2. Click **Export Modified BSP**.
3. In the file browser, look for **Lightmap export** in the export options.
4. Choose:
    - **Classic BSP29** — safest compatibility; keep this for now.

- **Enhanced BSPX (LMSHIFT)** — higher-density lightmaps; requires a compatible engine.

Our next test

1. Install v0.14.3.
2. Load the test map again using **Browse Game Maps**.
3. Try exporting in Classic mode.

If export still fails, send the new console output. The current log only identifies the UI icon problem, not the underlying export failure. Pasted text(20261108-160619).txt

We'll tackle the actual exporter once the interface is drawing cleanly.

‹/›

You said:

Ok ill get something to drink an then i test. While i do that why dont you think about how a redesign of the ui would look. Its a pretty big thing now. I guess the little sidepanel is not the best way of interfacing this. But i wanna hear your thoughts about how to make this more user friendly.

Worked for 2m 13s

ChatGPT said:

My main recommendation: **QSS should evolve from a collection of Blender tools into a small, workflow-oriented Quake editing environment inside Blender.** I agree that the current N-panel is becoming too cramped for what the add-on does.

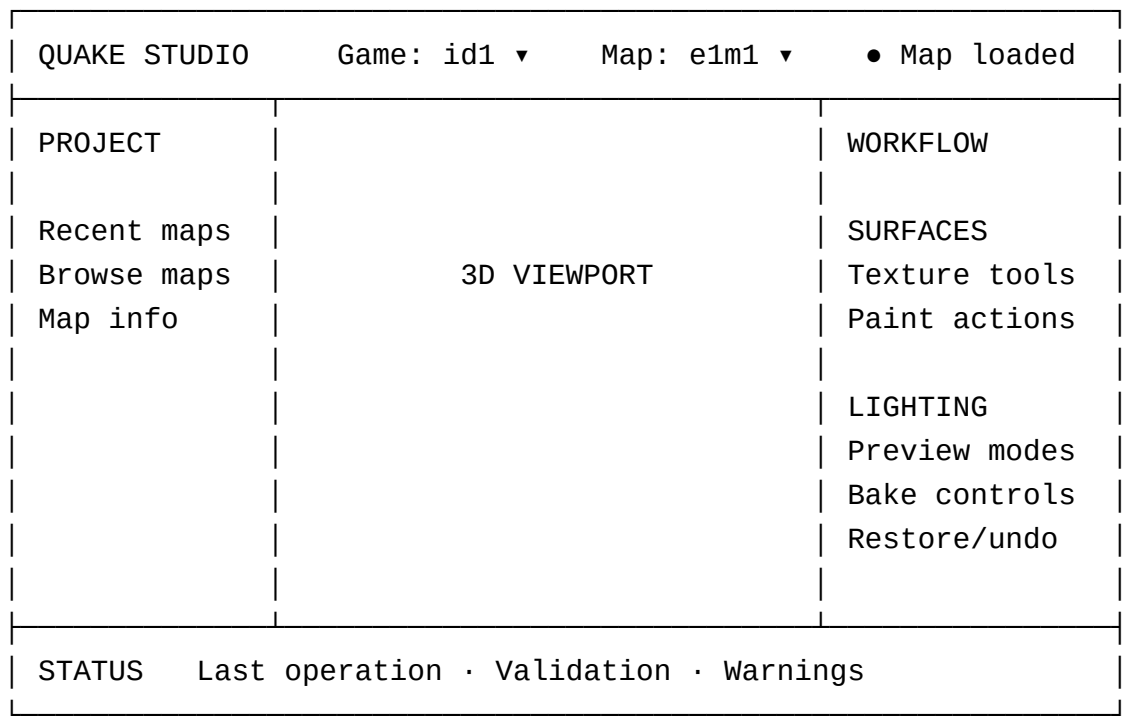
I looked at the current UI implementation. It has six main sections—Project, Navigation, Texture Workshop, Lighting Studio, Diagnostics and Export—with four additional panels nested under Lighting. There is already a sensible workflow hiding in there, but the interface mostly exposes features as lists of buttons and collapsible panels. That makes QSS increasingly powerful, but not necessarily easier to use.

I would redesign the *information architecture* before spending time on colours, icons or visual polish.

1. Give QSS its own workspace

I would introduce a dedicated **Quake Studio** workspace in Blender. The viewport stays central, because looking at and editing the map is the whole point. The QSS controls get a larger, more stable home rather than trying to fit everything into a narrow sidebar.

Conceptually:



That's the *design concept*, not a claim that Blender lets us build an arbitrary custom application window. I'd implement it using Blender-native workspaces, editors and add-on panels. The existing N-panel can remain as a compact quick-access area.

2. Organise the interface around tasks, not internal features

I would reduce the main navigation to four areas:

Area	What belongs there
Project	Quake installation, game/mod, map browser, import and current-map information
Surfaces	Face selection, texture editing, detach/make-paintable, texture painting
Lighting	Original/live/edited previews, Quake lights, lightmap painting, baking and recovery

Export	Export profile, destination, preflight checks, export and validation

A separate **Advanced / Developer** area would contain the Python test suite, batch map validation, detailed diagnostics and experimental Cycles baking.

That separation matters. A person who just wants to paint a wall shouldn't have to understand the test suite, and someone exporting a map shouldn't need to hunt through the lighting panels to find the relevant format setting.

3. Fix the confusing workflows first

A few changes would have an immediate usability payoff.

Map switching: the map browser should become a first-class project control. Show the active map in the header, provide a clear **Switch Map…** action, and offer an explicit choice to replace the current map or keep both. No more using Ctrl+Z to get rid of a map. The current browser already discovers loose BSP files and maps inside PAK archives; it mainly needs a better user-facing workflow.

Lighting preview: replace the separate Original, Live and Baked subpanels with one obvious mode selector:

```
Original · Live · Edited
```

Then put the controls for the selected mode underneath. The current A/B/C structure exposes the implementation rather than the decision the user is trying to make.

Export: this one is especially important given the confusion you just encountered. Clicking **Export BSP…** should first show a clear export-settings dialog:

- **Classic BSP29** — recommended, broad compatibility.
- **Enhanced BSPX** — higher-density lightmaps, requires a supporting engine.

Then choose the output path. After export, QSS should run its integrity validator automatically and display a clear pass/fail result. The setting should not feel like a hidden option in a file browser.

Texture editing: distinguish ordinary texture painting from lightmap painting at the top level. Show the selected-face count and the current painting target. Explain the effect of making faces independently paintable before doing it, rather than expecting users to know

what “detach faces” means.

4. Keep the good parts of Blender

I wouldn't try to build a completely independent application UI inside Blender. QSS depends heavily on Blender's viewport, mesh selection, image painting, materials and scene objects. Replacing those interactions would be a lot of work for little benefit.

Blender's Python API does support add-on panels in the Properties editor, and workspaces are layouts made from existing editors. It doesn't support creating entirely new editor-space types. That points toward a native **Quake Studio workspace**, with the main QSS controls in a wider Properties-editor panel and a compact N-panel for quick viewport operations.

Blender Pyt... +2

I'd keep these in the N-panel:

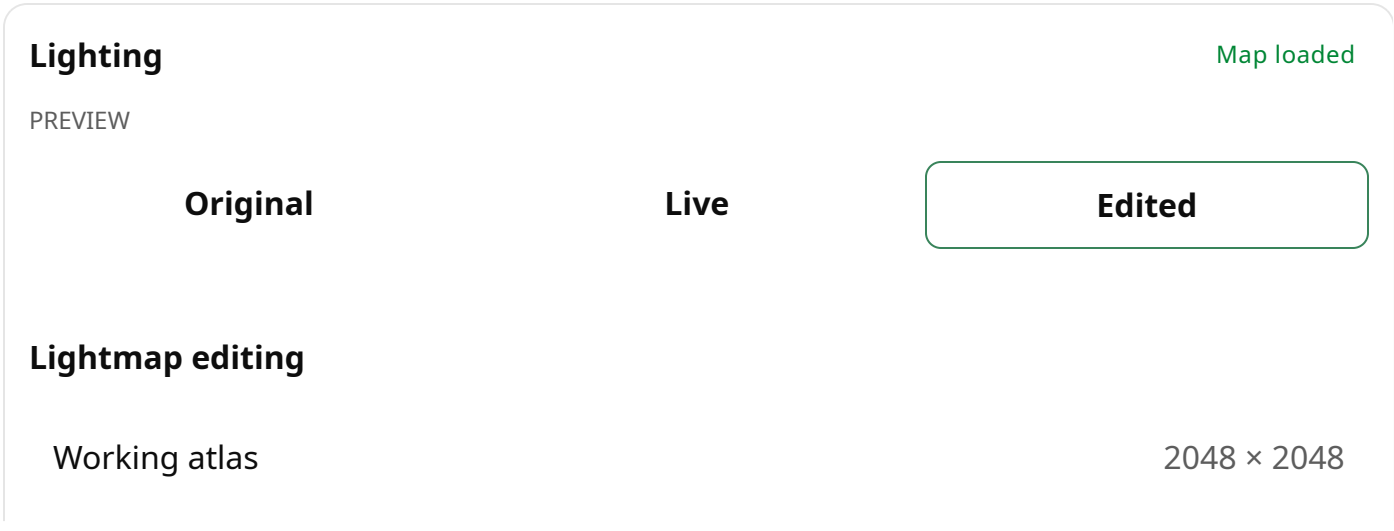
- Frame map / frame selection.
- Isolate and reveal faces.
- First-person navigation.
- A few quick painting and preview actions.

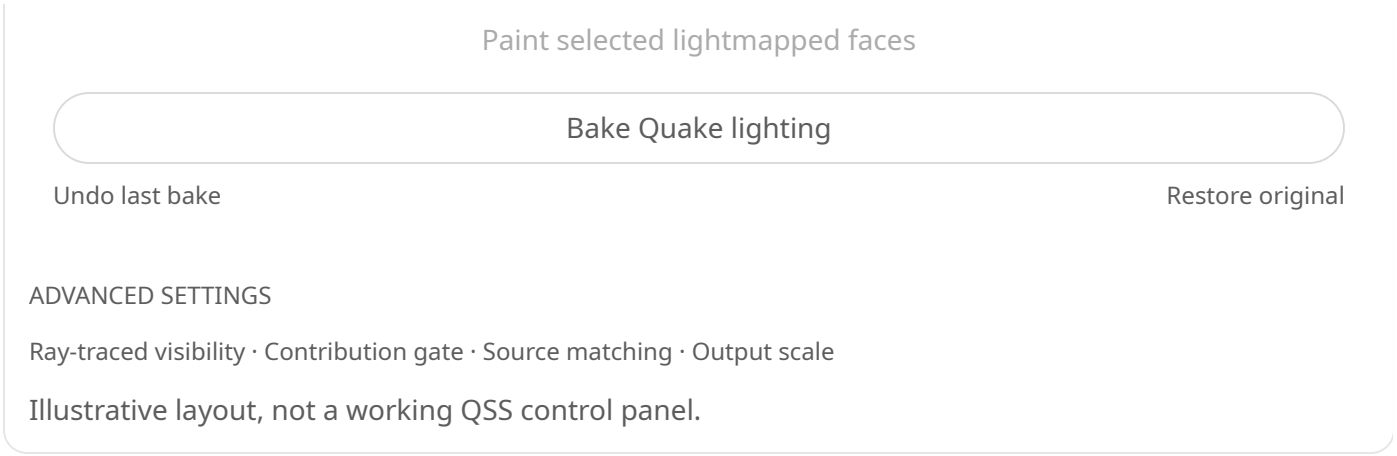
The main workflow controls would live in the larger workspace.

5. Make the interface progressive

The default view should show the controls most people need. Specialist settings should be available, but not compete for attention.

For example, Lighting could look like this:





This is the principle I'd apply everywhere: make the important action obvious, show the current state, and put technical parameters behind an expandable section.

6. How I'd approach the redesign

I would deliberately avoid rewriting all the underlying tools at once.

1. **Usability fixes:** explicit map switching, clearer map selection, an understandable export dialog and automatic export validation.
2. **Workspace prototype:** create the Quake Studio layout and move the main controls into a larger editor area. Keep the old N-panel available as a fallback.
3. **Workflow redesign:** rebuild Project, Surfaces, Lighting and Export around user tasks. Move developer tools out of the everyday workflow.
4. **Polish:** contextual help, meaningful warnings, clearer progress reporting and consistent terminology.

The existing processing code should remain largely untouched during the first UI pass. That keeps UI regressions separate from BSP import, texture editing and lightmap export bugs.

My overall take

QSS is becoming more than a texture-paint add-on. It is turning into a Quake map editing, lighting, validation and export environment. The interface should reflect that.

I'd aim for something that feels like a specialised Blender workspace for Quake—not a giant collection of add-on buttons, and not a separate application bolted onto Blender.

And I'd prioritise **map management and export clarity** first. Those are two places where the current interface has already made the workflow harder than it needs to be.

Once your v0.14.3 test is done, I'd turn this concept into a concrete UI prototype before migrating all the existing controls.

You said:

ok export works
the map runs in ironwail
tell me how to proceed and where i find controls i have not already used.
also just a little thing, i dont even know if you control it, but the open and save file dialgues the addon invokes are a bit small. I always have to resize them by hand. :D

Worked for 10s

ChatGPT said:

Excellent! That's a significant milestone: **QSS exports a BSP that runs in Ironwail.** We now have a working end-to-end path: import → edit → export → load in an actual Quake engine.

I'd move from “does it work?” testing to **feature validation** now. The goal is to exercise the controls you haven't used yet, one group at a time, while keeping the known-good test map untouched.

A sensible next test sequence

First, save a copy of your current Blender file and keep the working exported BSP as a baseline. For each test, change one thing, export to a new filename, and check it in Ironwail.

1. Start with navigation and face selection

All of these are in **3D Viewport → N → Quake → 02 · Navigation & Selection**.

Low risk

Control	What it does	How to test
Focus Selected Face(s)	Frames selected geometry in the viewport	Select a few faces and click it
Isolate Selected Faces	Hides everything except selected faces	Select a wall section and isolate
Reveal Hidden Faces	Unhides geometry	Reveal the hidden faces again

Optimize View Clipping	Adjusts viewport clipping for large maps	Try it when distant geometry disappears
FPS / Walk Navigation	Starts first-person map navigation	Walk around the test map
Hold MMB FPS Navigation	Enables hold-middle-mouse navigation	Enable it, hold MMB, use WASD

For FPS navigation, try the player-start button first if the map has an `info_player_start` entity. It should put you in the intended starting position.

Also try changing **Speed (Quake u/s)** and **Fast multiplier**. These are useful for navigating larger maps.

2. Try texture editing

Go to **03 · Texture Workshop**.

Use a copy of the map

1. Select the BSP mesh.
2. Enter Edit Mode.
3. Select a few faces.
4. Click **Detach Selected Faces**.
5. Inspect the resulting face-local paint canvas.
6. Click **Enter Texture Paint** and try painting.

The important distinction: detaching faces creates an independent 0–1 paint canvas. It is useful when you want to paint a particular face without modifying a shared texture used elsewhere.

After painting, export to a new BSP and check the result in Ironwail. Also inspect the texture name field (**BSP texture name**) in the workshop.

3. Explore lighting

Go to **04 · Lighting Studio**. This is where QSS starts getting particularly interesting.

Recommended next experiment

Try these in order:

A. Compare the lighting modes

Open the nested panel **A · Reference / Live / Baked**.

- **Original BSP Lightmap:** shows the imported lighting.
- **Live Lights:** previews the imported Quake light entities as Blender lights.
- **Baked / Working:** displays the current working lightmap.

Compare the same room in all three modes.

B. Inspect imported Quake lights

Open **B · Quake Light Entities**.

You can inspect and modify:

- `delay`
- `style`
- `wait`
- `anglescale`
- `_falloff`

These are Quake-specific light properties. Start by inspecting them; don't change all of them simultaneously.

C. Try the Quake lightmap solver

Open **C · Quake Direct Lightmap Solver**.

This is the feature I'd most like you to test next.

1. Select the map object.
2. Open the solver options.
3. Start with the existing defaults.
4. Click **Bake Quake Direct Lighting**.
5. Switch to Baked / Working preview.
6. Compare against Original.

The solver has controls for ray-traced visibility, contribution gate, source matching and output brightness.

If the result is worse, use **Undo Last Bake** or **Restore Imported BSP Lightmaps**.

D. Leave Cycles for later

The nested **D · Cycles PBR Bake (Experimental)** panel offers a different lighting approach. It uses Blender's path-traced baking, rather than QSS's Quake-specific direct-light solver.

I'd leave this until we've established the behaviour of the Quake solver.

4. Test diagnostics and validation

Go to **05 · Diagnostics & Recovery**.

These controls are particularly useful for us while developing QSS:

Control	Purpose
Save Lighting Diagnostics	Writes a detailed lighting report
Run Full Self-Test + Delay Benchmark	Tests the lightmap solver and Quake delay behaviour
Repair Lightmap Material Links	Attempts to repair preview/material connections

There are also testing tools in **01 · Project & Source**:

- **Validate Exported BSP...** compares an export with its source.
- **Validate Maps in Active Game** scans the maps in your selected game/mod.
- **Run QSS Python Test Suite** runs the standalone test suite inside Blender.

I would run the Python test suite now, and save the report if it passes.

5. Try enhanced export last

Under **06 · BSP Export**, export as you did before, but this time inspect the **Lightmap export** setting in the file browser's export options.

- **Classic BSP29:** what you've already tested successfully.
- **Enhanced BSPX (LMSHIFT):** higher-density per-face lightmaps, requiring engine support.

Ironwail is a sensible target for checking compatibility, but let's first test a simple map with enhanced export and compare it visually against the classic version. Keep the classic export as your known-good baseline.

About those tiny file dialogs :D

You're not imagining it. The standard import and export dialogs are Blender's native file browser, invoked through Blender's `ImportHelper / ExportHelper` mechanism.

The important limitation: Blender's documented `fileselect_add()` API doesn't expose a width or height argument. So QSS cannot simply say “open this file browser at 1200 × 800.”

Blender Pyt... +1

There are two possible approaches:

1. **Keep Blender's native browser.** This is the safest and most consistent option. We can improve the surrounding QSS dialogs and remember useful directories and filenames.
2. **Build a custom browsing workflow.** More control over layout, but significantly more code and more maintenance. I wouldn't replace Blender's browser just to solve its initial size.

I'd keep the native browser for now. The small map-selection popup is something we can improve independently.

What I want you to test next

My priority order would be:

QSS feature test checklist

0 / 10 tested 0%

- [ ] Focus / isolate / reveal faces
- [ ] FPS navigation and player start
- [ ] Detach faces and texture painting
- [ ] Original / Live / Baked lighting comparison
- [ ] Inspect imported Quake light properties
- [ ] Quake Direct Lightmap Solver
- [ ] Undo bake and restore original lightmap

- [ ] Run Python Test Suite
- [ ] Validate exported BSP
- [ ] Enhanced BSPX export in Ironwail

Reset checklist

My suggestion: start with the lighting preview comparison, then try the Quake Direct Lightmap Solver. That's a major part of what makes QSS different from a conventional BSP texture editor.

And once you've tried those, I can use your feedback to prioritise the next UI redesign and implement the larger, more comfortable dialogs where Blender gives us room to do so.

You said:

are those not tests that could be automated? The whole thing is controllable via the script editor. It could try everything with every map in the current map folder and collect the data for you to analyze and use to improve the app.

Worked for 18s

ChatGPT said:

Yes. In fact, **that is a much better testing strategy** than asking you to click through every feature manually.

If QSS is controllable through Blender's Python API, we can build an automated test runner that treats Blender as the test environment, the installed Quake maps as a test corpus, and the resulting reports as data for development.

I'd make this a proper **QSS Integration Test & Map Audit system**, not just a longer version of the existing Python unit-test suite.

The key distinction: we can automate most operations and verify their results, but we should distinguish between *a command that ran without an exception* and *a result that is actually correct*. Each test needs assertions and before/after measurements.

What the automated test run should do

I would build it in layers.

1. Discover and import every map

Use the currently selected Quake game/mod directory and enumerate:

- Every loose `maps/*.bsp`
- Maps contained in `pak*.pak` archives, if we want the full game corpus
- Map names, file sizes and source archive

Then import each map through QSS, just as a user would.

2. Exercise the actual QSS features

For each imported map, automatically test:

Area	Automated checks
BSP import	Geometry, face count, materials, textures, entities, lights and lightmaps
Navigation	Player start detection, selection operations, hide/reveal behaviour
Texture Workshop	Face detachment, material assignment and texture references
Lighting	Original/Live/Baked preview switching, light entity properties and energy calculations
Quake light solver	Bake, image validity, pixel statistics, undo and restore
Export	Classic BSP29 and Enhanced BSPX export
Validation	Run the existing export validator against every output
Recovery	Check that restoring the original data actually restores it

For destructive tests, the runner should work on temporary copies and never overwrite the user's original maps.

3. Collect useful measurements

This is where it becomes particularly valuable for development. For each map, collect:

- Import/export duration and memory usage where measurable

- Number of faces, vertices, materials, textures and lights
- Missing or unresolved textures and resources
- Lightmap dimensions, sample counts and brightness statistics
- Bake duration, output statistics and errors
- Export validation results
- Exceptions, warnings and full tracebacks
- Blender version and QSS version

The report should be machine-readable JSON, plus a readable HTML or text summary.

4. Separate genuine integration tests from UI tests

There is one important technical distinction: some QSS operators need a real 3D View context. A background Blender process cannot reliably test every viewport operation.

I'd therefore provide two modes:

Map corpus test Mostly headless

Imports, analyzes, bakes where supported, exports and validates every map. Suitable for repeatable runs and CI.

Full Blender UI test Script Editor

Runs the viewport-dependent operations in your normal Blender session, with actual editor and selection contexts.

Ironwail runtime compatibility can be a third layer: launch the engine against exported maps if its executable and a suitable test configuration are available. A successful BSP validation is not, by itself, proof that every engine will render the map correctly.

What I'd build next

A new **QSS Automated Map Test Runner**, accessible from the existing Diagnostics panel and runnable from Blender's Script Editor.

The workflow would be:

1. Select the Quake game/mod folder.
2. Click **Run Full Map Audit**.

3. Let QSS process all discovered maps, one at a time.
4. Automatically clean up temporary scenes and test files.
5. Save a report folder containing per-map results, aggregate statistics and failures.

I'd also have it continue after an individual map fails. One broken map should not abort a test of 200 maps.

One important design decision: the runner should test the real public QSS operators wherever possible, not merely call internal helper functions. Otherwise we risk testing the implementation while missing bugs in the actual user workflow.

I think this is the right next development step. It turns your Quake installation into a repeatable regression-test library, and gives us concrete evidence for deciding what to fix or improve instead of relying on occasional manual testing.

You said:

Very good! Implement it. Also since we have a config file now there are some more tools that may be helpful for the testing and whose location could be added to the config. namely the ericw tools, the quake executable in this case ironvail which can run demos and has a developer mode with detailed output. Impress me!

Worked for 13m 5s

ChatGPT said:

Implemented. QSS v0.15.0 now has a full automated map-audit system and external Quake-tool integration. 😁

The changes are committed to the existing feature branch and the draft PR. `main` remains untouched.

QSS v0.15.0 CI passed

120 / 120

Standalone tests passed · Python compilation passed · Blender install ZIP built

Download QSS v0.15.0 install ZIP ↗

View successful CI run ↗

View updated draft PR #1 ↗

What's new

1. Full Game Map Audit

Available under **N → Quake → Project & Source → Full Map Audit**.

It works without an imported map. It discovers loose BSPs and PAK-contained maps, then processes them individually.

For each map it tests:

- BSP structure, references and lightmap bounds.
- Actual QSS import and mesh metadata.
- Texture and lightmap UVs, materials and images.
- Original, Working and Live lighting previews.
- Original lightmap immutability.
- Lightmap material-link repair.
- Face selection, focus, clipping and hide/reveal.
- Player-start navigation.
- Face detachment and texture-paint activation on a disposable duplicate.
- Lightmap-paint activation.
- Classic BSP export and source-to-export validation.

Optional tests include Quake Direct Lightmap baking, Cycles baking with exact undo verification, and Enhanced BSPX export.

The runner continues after individual map failures and checkpoints its progress after every map. Press **Esc** to stop after the current map.

2. ericw-tools integration

In Project & Source, configure the folder containing:

- `qbsp`
- `vis`
- `light`
- `bspinfo` (if available)

QSS can then run:

- `bspinfo` on discovered maps.
- Verbose `light` runs on disposable BSP copies.
- An optional full `.map` → `qbsp` → `vis` → `light` compile pipeline where sibling MAP files exist.

The external commands use documented compiler options, including verbose output and game-directory selection. ericw-tools ... +2

Important: ericw `light` overwrites its BSP input. QSS therefore never gives it your original game BSP.

3. Ironwail integration

Configure the executable and optionally a demo file.

The new **Launch Ironwail...** command can:

- Load the currently selected QSS map.
- Play a selected `.dem`.
- Run a timedemo.
- Enable `developer 1`.
- Enable `-condebug`.

Each launch receives its own logging directory under Blender's QSS user data. QuakeSpasm documents `-condebug` console logging, which is inherited by the Ironwail-based workflow. GitHub +1

4. Reports for development

Blender user data / qss / audits / Map_Audit_timestamp

Each audit produces:

File	Contents
`report.json`	Complete machine-readable results
`report.txt`	Readable summary and failures
`progress.json`	Per-map checkpoints

Per-map folders	Exports, compiler logs and diagnostics

The report is also available in Blender's Text Editor as `QSS_Map_Audit_Report` and `QSS_Map_Audit_JSON`.

Your original BSPs are not modified by the audit. Temporary Blender objects and test copies are cleaned up, and the original viewport state is restored.

How to try it

1. Install the v0.15.0 ZIP linked above.
2. Open **N** → **Quake** → **Project & Source**.
3. Check that your Quake installation and active game/mod are configured.
4. Set the ericw-tools directory and Ironwail executable.
5. Start with the optional expensive tests disabled.
6. Click **Run Full Game Map Audit**.

Then try the ericw relight and compile options. Leave the full Quake and Cycles baking options for a later, more expensive run.

Runtime validation still needed

The CI suite has passed, but I haven't executed this new integration runner inside your actual Blender 5.2 installation. Viewport-dependent tests also need a normal Blender UI session; modal FPS navigation is explicitly marked as interactive rather than falsely reported as tested.

The next useful step: run the audit against your active game folder and send me the resulting report folder (or a ZIP of it). I'll analyze the failures, identify recurring map-specific patterns, and use that evidence to prioritize the next QSS fixes.

You said:

can you access our previous conversations (Quake Lightmap Workflow 1 - 4?

ChatGPT said:

I can use the context available in this conversation, but I **cannot automatically open or browse your entire ChatGPT conversation history**.

I don't currently have the full contents of **Quake Lightmap Workflow 1–4** available. I do have some carried-over QSS development context, but that is not a substitute for those conversations.

There are two good ways to bring them in:

1. **Export or upload the four conversations** as text, Markdown, HTML, or PDF. I can then read them together.
2. **Copy their contents here**, one conversation at a time.

I'd recommend providing all four. I can then reconstruct the project's technical history, including:

- The original lightmap workflow and design decisions.
- Algorithms, experiments and problems already investigated.
- Decisions about Quake lighting, styles, surface sampling and light entities.
- Features implemented versus planned.
- Outstanding issues and ideas that shouldn't be lost.

That would give us a much stronger technical foundation for the next QSS development phase, without accidentally repeating experiments or reversing earlier decisions.

You said:

I added our full chat history to the github on your branch and i will update it everytime we swicht chat. Feel free do add your own notes and documentations in that folder if you find something you want to remember later or deem a good idea to document. I will now go inspect 0.15.0 now and report afterwards.

Worked for 19s

ChatGPT said:

Excellent. I've confirmed that the `ai chat history` folder is present on our feature branch. It contains:

- Quake Lightmap Workflow **(0) through (5)**
- Setup Quake Mod Environment

That gives me a persistent project history I can consult when we switch chats. I'll treat those conversations as primary historical context, cross-check them against the current code, and add concise technical notes when they capture decisions, discoveries, constraints, or lessons that would otherwise be easy to lose.

I'll also keep documentation changes on our feature branch. `main` **stays untouched, and I won't merge anything without your approval.**

For now, go put v0.15.0 through its paces. I'm particularly interested in the Full Game Map Audit results and any unexpected Blender-state changes. When you report back, include the audit report or logs if available, and we'll work through the findings. Good luck!

You said:

v0.15.0: Error when actviating after install:
name '_on_external_tool_path_changed' is not defined

Worked for a few seconds

ChatGPT said:

Found the cause and fixed it on the feature branch.

Root cause: In `addon.py`, three Blender `StringProperty` definitions referenced `_on_external_tool_path_changed` as their update callback, but that function was never defined. Blender therefore raised a `NameError` during add-on registration.

Fix committed:

- Added the missing callback.
- It persists changes to the ericw-tools directory, Ironwail executable and optional demo path in QSS's existing user configuration.
- It safely ignores callbacks without a valid Scene context.

Committed `c66f7bbb`

The fix is on `fix/v0.11.1-metadata-style-order-test`; `main` remains untouched.

Please download/rebuild the add-on from the updated feature branch and try enabling it again. This addresses the reported registration error; I haven't yet verified it in a live Blender 5.2 session.